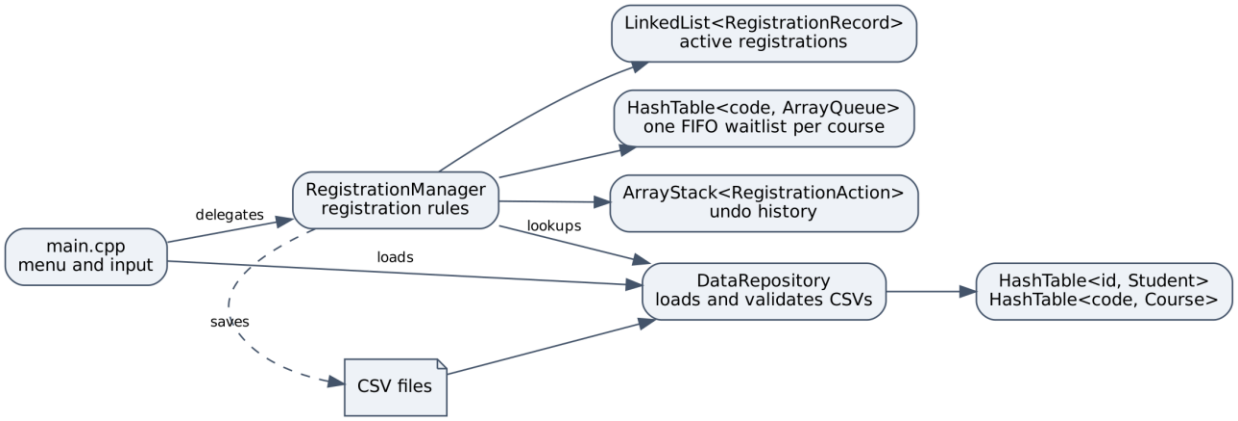
Assignment 2: Design and Performance Report

University Registration & Waitlist System | CPSC 316 Data Structures | Paxton Dutt

1. Architecture

The program keeps data storage, registration rules, and user interaction separate. main.cpp reads and checks menu input, then passes the work to the classes:

- DataRepository loads and validates the CSV files and holds the student and course tables.
- RegistrationManager holds the active registrations, waitlists, and undo history, and applies the registration rules.
- Student and Course store their own records, including completed courses and prerequisite codes.



Visual representation created with Graphviz

2. Why a queue for waitlists and a stack for undo

A waitlist must be fair, so the student who has waited longest gets the next eligible seat. This is first-in, first-out (FIFO) behavior, which is what a queue does. A circular array removes the front element in constant time without shifting the others, and it reuses freed slots by wrapping indexes with the modulo operator. When the array is full, it doubles in size and copies the elements over in queue order, so FIFO order stays correct even if the contents had wrapped past the end of the array.

Undo must reverse the most recent action first. This is last-in, first-out behavior, which is what a stack does. Undo edits the registration list directly instead of calling registerStudent or dropStudent, so reversing an action never creates another undo record.

3. Hash table design

The table handles collisions with separate chaining: each bucket holds a chain of key-value entries, and keys that map to the same bucket share a chain. To look up a key, the table hashes it with std::hash, takes the result modulo the bucket count, and compares actual keys along that chain. Separate tables map student IDs to students, course codes to courses, and course codes to waitlists.

The rehash threshold is 0.75, meaning the table grows before an insert that would push the load factor above 0.75. This keeps the expected chain length below one entry per bucket, so lookups stay fast without leaving most buckets empty. Java's HashMap uses the same default, and its documentation describes it as a good tradeoff between time and space costs [3].

The table grows to 2n + 1 buckets, which keeps the count odd and roughly doubles capacity. Every key is rehashed, and chain nodes are moved with std::list::splice, so pointers returned by find() remain valid after a rehash.

Observed load factors, from a test that inserted 1,000 keys into a table starting with 2 buckets:

Buckets before	Buckets after	Load before insert	Load after insert
2	5	0.500	0.400
5	11	0.600	0.364
11	23	0.727	0.391
95	191	0.747	0.377
767	1,535	0.750	0.375

After loading the provided data, the student table holds 1,500 entries with a load factor of 0.651, and the course table holds 80 entries with a load factor of 0.559. Both started at 17 buckets. All 1,000 test keys were still found with the correct values after the final rehash.

4. Complexity

Here n is the number of active registrations, w is the length of one waitlist, N is the number of entries in a hash table, and p is the number of prerequisites for a course.

Operation	Time	Explanation
Register	O(n + w + p)	Hash lookups are O(1) on average. Duplicate and enrollment checks scan the list, and waitlist duplicate checks scan the queue.
Drop	O(n), or O(wn) worst case	Removal scans the list. Each waitlist entry examined needs an O(n) registration check.
Hash find / insert / erase	O(1) average, O(N) worst	Average assumes short chains from a bounded load factor. An insert that triggers a rehash costs O(N).
Waitlist enqueue	O(1) amortized, O(w) worst	The worst case occurs when the array must grow.
Waitlist dequeue	O(1)	Moves the front index with no shifting.
Linked-list search / remove	O(n)	Both walk the list from the head. Inserting at the front is O(1).

5. Tested edge cases

Automated tests in tests/ cover all twelve required edge cases. Four notable results:

Case	Test	Result
Waitlisted student becomes ineligible	Students 200002 and 200003 wait for PHYS165. 200002 is made inactive, then a seat opens.	200002 is removed from the waitlist with a reason, and 200003 is promoted. Enrollment stays at 30/30.
Undoing a drop after promotion	200921 drops PHYS165 and waitlisted 200001 takes the seat. The drop is then undone.	Undo reports that PHYS165 has no open seat. Capacity is not exceeded, and the action is consumed.
Over-capacity starting data	One student drops ENGR360, which starts at 35/20, while another is waitlisted.	Enrollment becomes 34/20. Nobody is promoted, and the waitlisted student keeps their place.
Queue wrap-around and growth	Fill a 4-slot queue past the end of its array, then add one more element.	Capacity grows to 8, and elements dequeue in the original order: 3, 4, 5, 6, 7.

6. Design decisions and assumptions

- Any recorded grade, including F, counts as completing a course, because the specification sets no minimum grade. I did initially consider not counting F's, but changed it later on.
- Initial registrations are kept as provided, even when a course is over capacity, a student is inactive, or a prerequisite is missing, because that file is the existing state. In the provided data, a decent amount of courses start over capacity.
- Joining a waitlist and being promoted do not create undo records, because undo covers a user's successful register and drop operations.
- Undoing a drop checks seat availability and duplicates, but not prerequisites, because some initial registrations do not meet them. A bug fix is what led to this decision.

References

[1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, Introduction to Algorithms, 4th ed. Cambridge, MA: MIT Press, 2022. Chapters on elementary data structures, hash tables, and amortized analysis.

[2] M. A. Weiss, Data Structures and Algorithm Analysis in C++, 4th ed. Boston, MA: Pearson, 2014. Chapters on lists, stacks, queues, and hashing.

[3] Oracle, "HashMap (Java Platform SE 8)," Java API documentation. https://docs.oracle.com/javase/8/docs/api/java/util/HashMap.html